AGENTIC PAYMENT RECOVERY

Repechage

Agentic Payment Recovery for Failed Razorpay Checkouts

AI reasons. Policy decides. Execution is controlled.

Every failed checkout is a customer who tried to pay you.

Repechage brings them back.

Every failed checkout is a customer who tried to pay you.

Repechage brings them back.

From failed payment to controlled recovery

The revenue loss is already in the funnel.

Customers are ready to pay. The checkout fails—and intent evaporates.

₹98,24,112
transaction value

66.6%
flagged at-risk • ₹65,38,889

438
genuinely recoverable events

card_declined • insufficient_funds • otp_timeout • network_error

1,000 failed-payment events

Same trigger. Same action. Wrong outcome.

Wastes attempts
One response for every failure

Annoys customers
Irrelevant, repeated actions

Misses recoveries
No diagnosis of the cause

Never knows when to stop
No explicit stopping criteria

Reason broadly.
Act narrowly.

AI reasons. Policy decides. Execution is controlled.

A bounded decision, not an open-ended agent.

Every decision → audit trail

01	DETECT	Deterministic rules on status + failure_reason
02	DIAGNOSE	LLM Recovery Agent reads rich transaction context
03	POLICY GATE	Safety and business rules hold final authority

```
recover_now
send_payment_link
wait_and_retry
escalate_to_merchant
stop
```

The policy gate is the authority.

WHEN IN DOUBT → ESCALATE, DO NOT EXECUTE

Attempts ≥ 3

STOP

Already recovered

STOP

Amount > ₹5,000

ESCALATE

Max 10 real actions / run

HARD CAP

Live execution

DEFAULT OFF

Invalid / low-confidence

GATED

Two models.
One hard boundary.

DETERMINISTIC DETECTOR

status • failure_reason • attempts
Explainable risk classification
Zero LLM cost

LLM RECOVERY AGENT

Rich transaction context
Diagnosis + action + confidence
Structured recommendation

POLICY GATE
und truth. Neither c

Structured in.
Auditable out.


```json
{
  "diagnosis": "card_declined",
  "recovery_probability": 0.82,
  "recommended_action": "send_payment_link",
  "reason": "New link can recover payment",
  "confidence": 0.91
}
```


SCHEMA-VALIDATED → POST-FILTERED → POLICY-GATED

AUDIT SURFACE

transactions
detection_results
agent_decisions
recovery_attempts
logs • webhooks • merchants

From merchant signal to accountable action.

PostgreSQL on Neon • 8 tables • Alembic migrations

FRONTEND	Dashboard • Analytics • Audit • Developers • Security
API LAYER	FastAPI • pages • auth • webhooks • dashboard
DECISION	Detect → LLM Agent → Policy Gate
DATABASE	Transactions + execution lifecycle

One failed checkout.
A controlled path back.

Recovery executes only when policy approves.

Razorpay failure event — Detect + diagnose — Policy approval — Bounded action — Dashboard + audit

Capture → reason → authorize → execute → explain

Safer interventions.
Lower total recovery.

73.0% AI HIT-RATE
vs 66.2% baseline

ENGINEERING LESSON / Precision improved, but fewer actions meant less net revenue. Optimize for the right opportunities—not simply fewer interventions.

THE NEXT RECOVERY SHOULD BE
EXPLAINABLE, SAFE, AND WORTH IT.

Recover revenue.
Keep control.

AI reasoning + strict policy control + transparent audit.

LET'S DISCUSS →